Tema 4. Estructures de dades homogènies: Arrays

Assignatura: Introducció a la programació
Grau d'Enginyeria Informàtica
Curs 26-27

Inmaculada Rodriguez Santiago
Departament de Matemàtiques i Informàtica



Contingut

4.1 Introducció als arrays

4.2 Arrays unidimensionals

4.3 Declaració, inicialització i accés

4.4 Operacions amb els elements

4.5 Esquemes iteratius amb arrays

4.6 Arrays com a paràmetres de funcions

4.7 Cadenes de caràcters

4.8 Funcions per cadenes

4.9 Arrays multidimensionals

4.1 Introducció als arrays

Un array és una variable que emmagatzema un conjunt de dades del mateix tipus com ara notes d'una classe, temperatures diàries o alçades d'un grup.

- Tots els elements comparteixen el mateix nom i s'identifiquen mitjançant un índex.

Perquè necessitem els arrays?

```
float notaA1;
float notaA2;
float notaA3;
float notaA4;
...
float notaAn;
```

```
#define N 100
float notesAlumnes[N];
```

Contingut

4.1 Introducció arrays

4.2 Arrays unidimensionals

4.3 Declaració, inicialització i accés

4.4 Operacions amb els elements

4.5 Esquemes iteratius amb arrays

4.6 Arrays com a paràmetres de funcions

4.7 Cadenes de caràcters

4.8 Funcions per cadenes

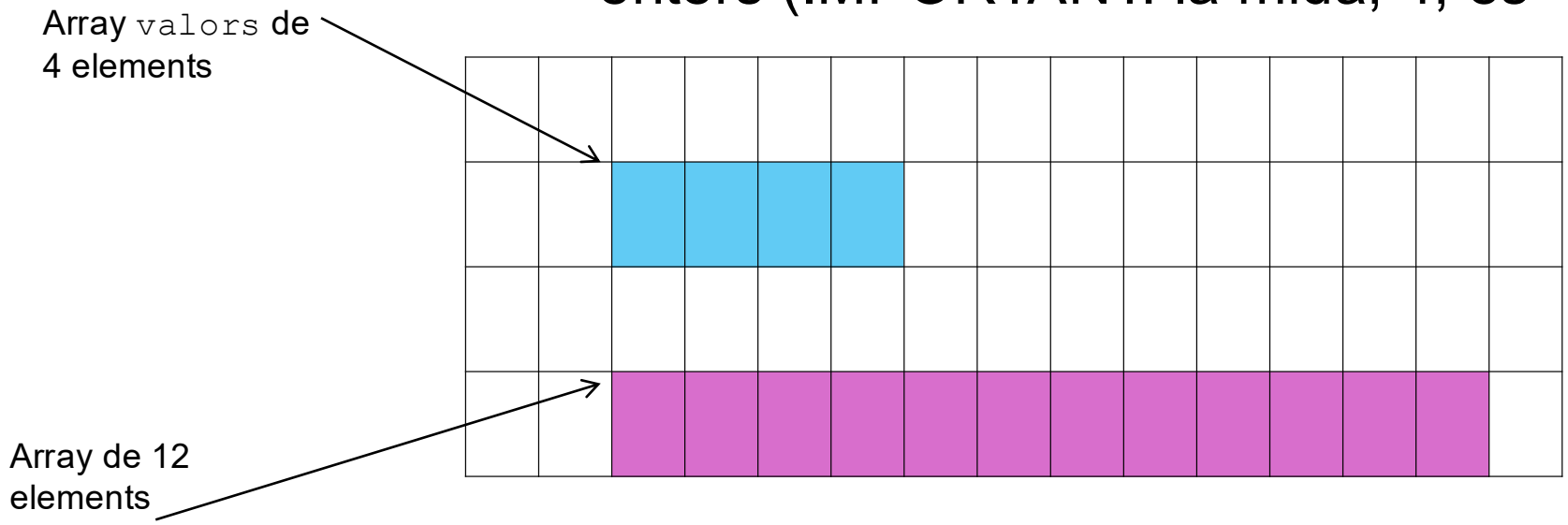
4.9 Arrays multidimensionals

4.2 Arrays unidimensionals

Conjunt d'elements del mateix tipus emmgazemats en posicions contigües de memòria, als quals s'accedeix mitjançant un únic índex (començant en 0). I

IMPORTANT: la seva mida mai canvia.

Exemple: `int valors[4];` El compilador reserva espai per a quatre enters (IMPORTANT: la mida, 4, és constant).



Contingut

4.1 Introducció als arrays

4.2 Arrays unidimensionals

4.3 Declaració, inicialització i accés

4.4 Operacions amb els elements

4.5 Esquemes iteratius amb arrays

4.6 Arrays com a paràmetres de funcions

4.7 Cadenes de caràcters

4.8 Funcions per cadenes

4.9 Arrays multidimensionals

4.3 Declaració, inicialització i accès

Declaració:

```
tipus nom_array[nombre_d_elements];
```

- El `tipus` indica quin tipus de dada conté cada posición de l'array
- El `nombre_d_elements` indica la capacitat de l'array, aquest nombre és constant.

Exemple: `float altures[100];`

Emmagatzema les altures dels alumnes d'una classe, màxim 100 altures.

`altures[0]` és el primer element i `altures[99]` és l'últim.

4.3 Declaració, inicialització i accès

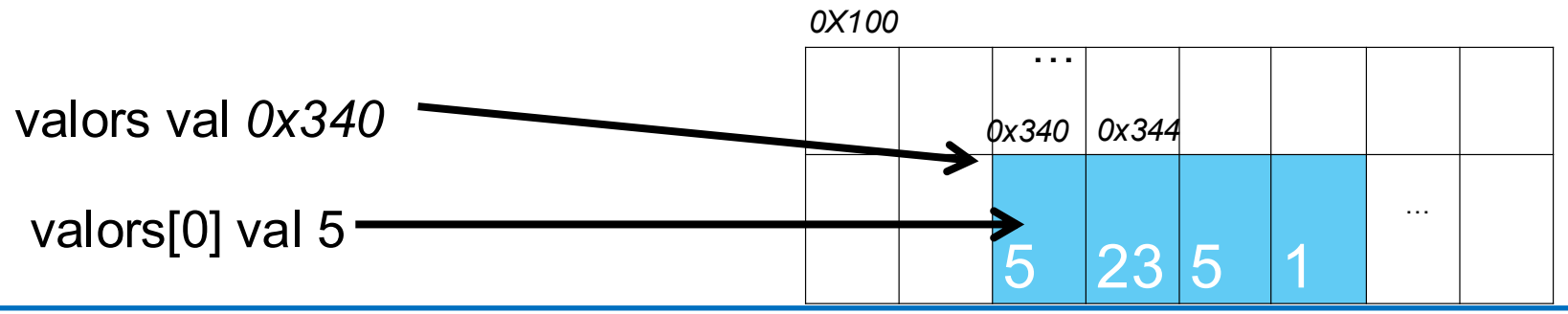
En declarar `tipus nom_array[nombre_d_elements]` es reserven `nombre_d_elements` posicions consecutives per a valors de tipus `tipus`.

- `nom_array` representa l'adreça del primer element de l'array.

  Atenció! Veurem que aquesta relació nom_array-adreça és important per entendre el pas d'arrays a funcions.

S'accedeix a un element amb el nom de l'array i el seu índex entre claudàtors: `nom_array[i]` és el valor emmagatzemat en la posició i

```
int valors[4] = {5, 23, 5, 1};
```



4.3 Declaració, inicialització i accès

Els elements es poden inicialitzar en el moment de la declaració:

```
int valors[4] = {5, 23, 5, 1};
```

Si s'omet la mida, el compilador la dedueix: `int valors[] = {14, 32, 44, 12};`

`int valors[6] = {5};` Posa el primer element a `5` explícitament.
Els elements restants,que no tenen valor, s'inicialitzen automàticament a 0.

En declarar un array sense inicialitzador, els valors NO queden inicialitzats automàticament. Si és una variable local conté "basura", si es global sí s'inicia a 0.

Contingut

4.1 Introducció als arrays

4.2 Arrays unidimensionals

4.3 Declaració, inicialització i accés

4.4 Operacions amb els elements

4.5 Esquemes iteratius amb arrays

4.6 Arrays com a paràmetres de funcions

4.7 Cadenes de caràcters

4.8 Funcions per cadenes

4.9 Arrays multidimensionals

4.4 Operacions amb els elements

Els elements es poden utilitzar com qualsevol variable del tipus corresponent.

Assignació: `altures[2] = 1.83;`

Còpia: `altures[3] = altures[2];`

Operació: `altures[4] += 0.20;`

Lectura: `scanf("%f", &altures[2]);`

Mida: `sizeof(array)` retorna la mida total de l'array, `array`, en bytes

`sizeof(<tipus>)` retorna la mida en bytes d'un element de tipus `<tipus>`

Nombre d'elements de l'array valors: `sizeof(valors)/sizeof(int)`

4.4 Operacions amb els elements

Exemple sizeof()

```c
#include <stdio.h>

int main() {

    float altures[5] = {1.70, 1.75, 1.80, 1.65, 1.90};

    //sizeof() retorna un valor de tipus size_t,
    //%zu és el format correcte per mostrar-lo.
    printf("\nMida total de l'array: %zu bytes\n",
           sizeof(altures));

    printf("Mida d'un element: %zu bytes\n",
           sizeof(float));

    int nombre = sizeof(altures) / sizeof(float);

    printf("Nombre d'elements: %d\n", nombre);

    return 0;
}
```

Sortida per pantalla:

```
Mida total de l'array: 20 bytes
Mida d'un element: 4 bytes
Nombre d'elements: 5
```

Exemple NO FER sizeof() d'un paràmetre array dintre d'una funció

```c
int demo_no_fer_sizeof_dins_funcio(float altures[]){
    // NO es pot calcular la grandària de l'array dins de
    // la funció ja que retornarà la grandària de un punter
    // (altures és un punter a float) i NO
    // la grandària de l'array que hem passat com
    // paràmetre actual
    printf("\nfuncio: Mida total de l'array: %zu bytes\n",
           sizeof(altures));

    printf("funcio: Mida d'un element: %zu bytes\n",
           sizeof(float));

    int nombre = sizeof(altures) / sizeof(float);

    return nombre;

}
```

```c
#include <stdio.h>

int  demo_no_fer_sizeof_dins_funcio(float []);
int main() {

    float altures[5] = {1.70, 1.75, 1.80, 1.65, 1.90};

    //sizeof() retorna un valor de tipus size_t,
    //%zu és el format correcte per mostrar-lo.
    printf("\nMida total de l'array: %zu bytes\n",
           sizeof(altures));

    printf("Mida d'un element: %zu bytes\n",
           sizeof(float));

    int nombre = sizeof(altures) / sizeof(float);

    printf("Nombre d'elements: %d\n", nombre);

    printf("Nombre d'elements ERRONI que retorna la funció: %d",
                        demo_no_fer_sizeof_dins_funcio(altures));
    return 0;
}
```

Sortida per pantalla:

```
Mida total de l'array: 20 bytes
Mida d'un element: 4 bytes
Nombre d'elements: 5

funcio: Mida total de l'array: 8 bytes
funcio: Mida d'un element: 4 bytes
Nombre d'elements que retorna funcio_grandaria: 2
```

Contingut

4.1 Introducció als arrays

4.2 Arrays unidimensionals

4.3 Declaració, inicialització i accés

4.4 Operacions amb els elements

4.5 Esquemes iteratius amb arrays

4.6 Arrays com a paràmetres de funcions

4.7 Cadenes de caràcters

4.8 Funcions per cadenes

4.9 Arrays multidimensionals

4.5 Esquemes iteratius amb arrays

Podem interpretar els arrays com a seqüències d'elements i per tant podem adaptar els esquemes de programació de seqüències:

- recorregut
- cerca

Caracterització de la seqüència dels índex d'un array:

```
Primer()= 0
Següent(x)= x+1
FiSeq(x)= x >= nombre_elements
```

4.5 Esquemes iteratius amb arrays

```c
#include <stdio.h>
#define TAMANY 10  // La grandària també pot ser una variable entera, n,
                   // però NO pot canviar durant l'execució, e.g .int array[n];
int main() {
  int array[TAMANY];
  /* 1)      * Identif:    indexos de l'array des de 0 a TAMANY-1
   * Primer:      idx = 0
   * Seg(idx):    idx++
   * FiSeq(idx): idx >= TAMANY
   * Recorregut     */
  for (int idx = 0; idx < TAMANY; idx++) {
    printf("Valor %d? \n", idx + 1);
    scanf("%d", &array[idx]);
  }
  //idem 1)
  for (int idx = 0; idx < TAMANY; idx++) {
    printf("Element %d = %d\n", idx + 1, array[idx]);
  }
  return 0;
}
```

4.5 Esquemes iteratius amb arrays

```
/* Donat un número per teclat, s'ha d'indicar si es troba a l'array, en cas afirmatiu
   mostrar la posició, en cas contrari mostrar missatge "Número no trobat". */
#include <stdio.h> #include <stdbool.h> #define TAMANY 4
int main() {
  int idx; bool trobat; int array[TAMANY] = {15, 42, 7, 30}; int num;
  printf("Quin número vols cercar?\n");
  scanf("%d", &num);
  /* Identif:    indexos de l'array des de 0 a TAMANY-1
     Primer:     idx = 0
     Seg(idx):   idx++
     FiSeq(idx): idx >= TAMANY
     Cerca Condició cerca: array[idx] == num    */
  idx = 0;
  trobat = false;
  while (idx < TAMANY && !trobat) {
    if (array[idx] == num)
      trobat = true;
    else
      idx++;
  }
  if (trobat) printf("Trobat a la posició %d\n", idx + 1);
  else        printf("Número no trobat\n");
  return 0;
}
```

Contingut

4.1 Introducció als arrays

4.2 Arrays unidimensionals

4.3 Declaració, inicialització i accés

4.4 Operacions amb els elements

4.5 Esquemes iteratius amb arrays

4.6 Arrays com a paràmetres de funcions

4.7 Cadenes de caràcters

4.8 Funcions per cadenes

4.9 Arrays multidimensionals

4.6 Arrays com a paràmetres de funcions

En la declaració de una funció s'indica el nom de l'array i el nombre d'elements de l'array. Recordeu que dintre de la funció no es pot calcular la grandària (veure diapositiva 13).

Per exemple:

```
void llegir_array(float dades[], int n);
```

Exemple de crida:

```
llegir_array(array, 5);
```

Els canvis que la funció faci sobre els elements afecten l'array original. Estem fent un pas per referència ja que el nom de l'array es la direcció on comença l'array.

La funció que rep un array també es pot declarar amb l'array com a punter, és equivalent a la declaración de dalt:

```
void llegir_array(float *dades, int n);
```

Exercici 

Escriu una funció que llegeix nombres decimals des de teclat i els emmagatzema en un array.

Exercici 

Escriu una funció que, a partir dels 6 dígits premiats de la loteria primitiva i els 6 dígits que una persona ha marcat a la seva butlleta, calculi el nombre de dígits que ha encertat.

Contingut

4.1 Introducció als arrays

4.2 Arrays unidimensionals

4.3 Declaració, inicialització i accés

4.4 Operacions amb els elements

4.5 Esquemes iteratius amb arrays

4.6 Arrays com a paràmetres de funcions

4.7 Cadenes de caràcters

4.8 Funcions per cadenes

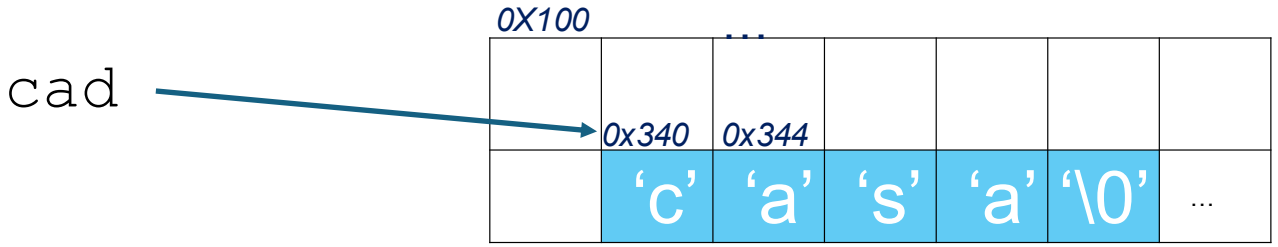
4.9 Arrays multidimensionals

4.7 Cadenes de caràcters

Una cadena de caràcters és un array de dades de tipus char.

Les cadenes en C acaben amb el caràcter especial fi de cadena '\0'.

Exemple: `char cad[] = {'c', 'a', 's', 'a', '\0'};`



També es pot fer:

```
char cadena[] = "casa";  //s'assigna espai per a 5 caràcters, inclòs '\0'
char cadena[20] = "casa"; //20 caràcters maxim, inclòs '\0'
```

4.7 Cadenes de caràcters

La funció `scanf("%s", cadena)` llegeix una cadena des de teclat fins que troba un espai en blanc. No llegeix un nom compost complet!

Per limitar els caràcters amb scanf, es pot indicar una amplada màxima, per exemple %19s en un array de 20 posicions.

```
1
2 #include <stdio.h>
3
4 int main() {
5
6     char cadena[20];
7
8     printf("Introduce tu nombre: ");
9     // Leemos como máximo 19 caracteres
10    scanf("%19s", cadena);
11
12    printf("Tu nombre es: %s\n", cadena);
13
14    return 0;
15 }
```

4.7 Cadenes de caràcters

La funció fgets(cadena, mida, stdin), permet llegir per teclat una cadena i controlar el nombre màxim de caràcters. stdin representa l'entrada estàndard (per teclat)

fgets pot conservar el salt de línia '\n'; es pot eliminar substituint-lo per '\0'.

```c
#include <stdio.h>
#include <string.h>

int main() {

    char cadena[50];

    printf("Introduce tu nombre completo: ");

    // Leemos como máximo 49 caracteres
    fgets(cadena, 50, stdin);

    // Eliminamos el salto de línea '\n'
    // strcspn() busca la posició de "\n" en cadena
    cadena[strcspn(cadena, "\n")] = '\0';

    printf("Tu nombre es: %s\n", cadena);

    return 0;
}
```

Contingut

4.1 Introducció als arrays

4.2 Arrays unidimensionals

4.3 Declaració, inicialització i accés

4.4 Operacions amb els elements

4.5 Esquemes iteratius amb arrays

4.6 Arrays com a paràmetres de funcions

4.7 Cadenes de caràcters

4.8 Funcions per cadenes

4.9 Arrays multidimensionals

4.8 Funcions per cadenes

https://en.cppreference.com/c/string/byte

Les funcions de cadena es troben a la biblioteca `<string.h>`.

`char *strcpy(char * desti, const char *origen);` copia la cadena sencera `origen` en `desti` (inclòs el '\0' final), i retorna `desti`.

`char *strncpy(char *desti, char *origen, int n);` copia com a màxim `n` caràcters de `origen` en `destí`, i retorna `destí`.

Atenció! Cal vigilar el fi de cadena. No sempre afegeix el '\0'.

- Si `origen` té n caràcters o més, `destí` es queda sense terminador i deixa de ser una cadena vàlida.
- Si `origen` és més curta que n, omple la resta de `destí` amb '\0' fins a completar n caràcters.

4.8 Funcions per cadenes

https://en.cppreference.com/c/string/byte

Exemple `strcpy` i `strncpy`

```c
#include <stdio.h>
#include <string.h>

int main() {

    char origen[30] = "Hola mundo";

    char destino1[30];
    char destino2[30];

    strcpy(destino1, origen);
    printf("Con strcpy: %s\n", destino1);

    // copiar como maximo 4 caracteres
    strncpy(destino2, origen, 4);
    // añadimos el caracter final manualmente
    destino2[4] = '\0';
    printf("Con strncpy: %s\n", destino2);

    return 0;
}
```

Sortida per pantalla

```
Con strcpy: Hola mundo
Con strncpy: Hola
```

4.8 Funcions per cadenes

https://en.cppreference.com/c/string/byte

`char *strcat(char * desti, const char *origen);` afegeix `origen` al final de `destí` i retorna `destí`.

`int strlen(char *cadena);` retorna la longitud de `cadena` sense comptar el caràcter '\0'.

```
 7      char cadena1[50] = "Hola";
 8      char cadena2[] = " mundo";
 9
10      int longitud = strlen(cadena1);
11      printf("Longitud de cadena1: %d\n", longitud);
12
13      strcat(cadena1, cadena2);
14      printf("Cadena concatenada: %s\n", cadena1);
15      longitud = strlen(cadena1);
16      printf("Nueva longitud: %d\n", longitud);
```

Sortida per pantalla

```
Longitud de cadena1: 4
Cadena concatenada: Hola mundo
Nueva longitud: 10
```

4.8 Funcions per cadenes

https://en.cppreference.com/c/string/byte

- `atoi()`, `atol ()` i `atof ()` converteixen cadenes a valors numèrics int, long i double
- `tolower()` i `toupper()` converteixen un caràcter a minúscula o majúscula
- En C standard no hi ha funció per convertir una cadena a minúcules o majúscules, `_strlwr ()` i `_strupr ()` són extensions (no standard) de windows

```
 8      char numero1[] = "25";
 9      char numero2[] = "100000";
10      char numero3[] = "3.1416";
11
12      printf("atoi: %d\n",   atoi(numero1));
13      printf("atol: %ld\n",  atol(numero2));
14      printf("atof: %.4f\n", atof(numero3));
15
16      char letra1 = 'A';
17      letra1 = tolower(letra1);
18      printf("tolower: %c\n", letra1);
19
20      char letra2 = 'b';
21      letra2 = toupper(letra2);
22      printf("toupper: %c\n", letra2);
```

Sortida per pantalla

```
atoi: 25
atol: 100000
atof: 3.1416
tolower: a
toupper: B
```

Contingut

4.1 Introducció als arrays

4.2 Arrays unidimensionals

4.3 Declaració, inicialització i accés

4.4 Operacions amb els elements

4.5 Esquemes iteratius amb arrays

4.6 Arrays com a paràmetres de funcions

4.7 Cadenes de caràcters

4.8 Funcions per cadenes

4.9 Arrays multidimensionals

4.9 Arrays multidimensionals

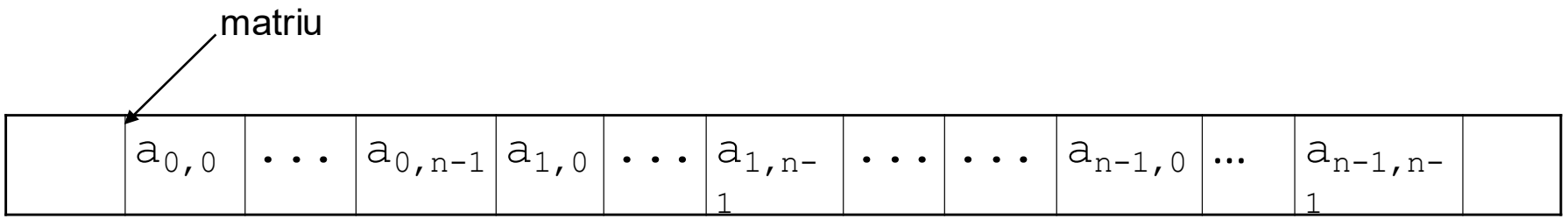
Un array bidimensional té dues dimensions: files i columnes.

Cada element s'identifica amb dos índexs: `a[i][j]`.

Els elements s'emmagatzemen en posicions consecutives de memòria, per files en C.

$$\begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & \cdots & a_{0,m-1} \\ a_{1,0} & a_{1,1} & a_{1,2} & \cdots & a_{1,m-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-1,0} & a_{n-1,1} & a_{n-1,2} & \cdots & a_{n-1,m-1} \end{pmatrix}$$

matriu amb n files m columnes

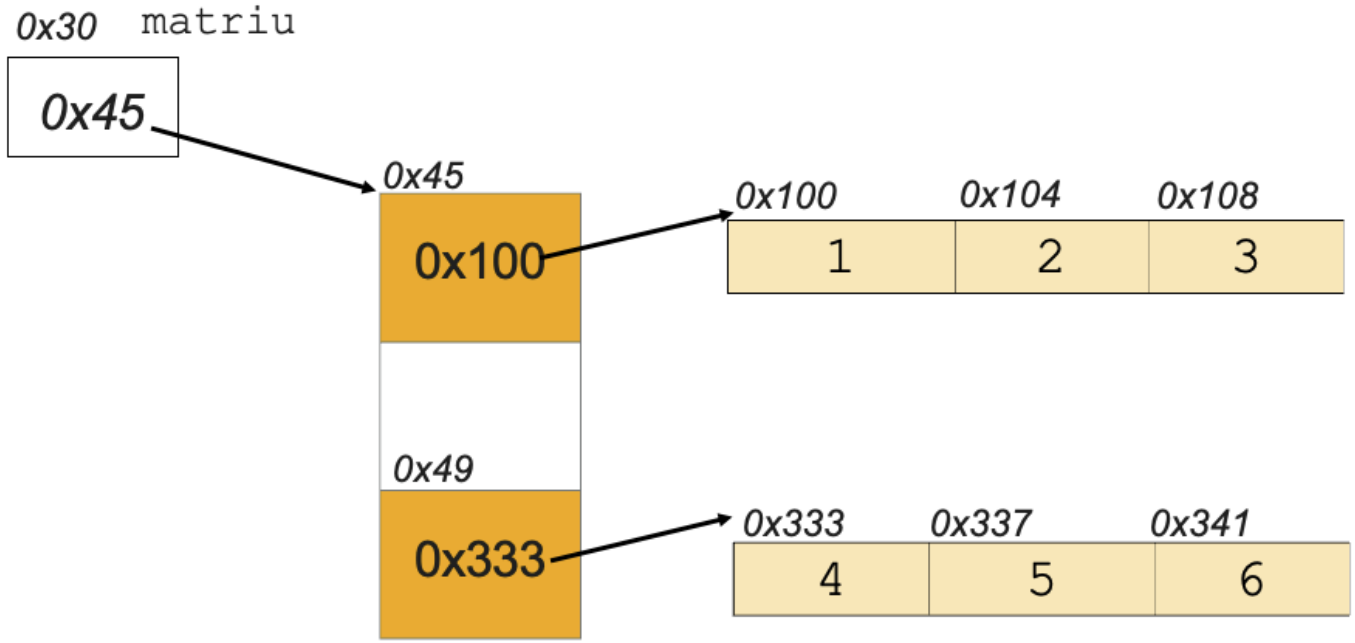


Exemple: `float matriu[32][100];` representa 32 files amb 100 valors cadascuna.

4.9 Arrays multidimensionals

Encara que s'emmagatzemen en posicions consecutives, per facilitar la comprensió els dibuixarem així. Aquesta representació ajuda a comprendre la relació entre arrays i punters.

```
matriu[2][3] ={{1,2,3},{4,5,6}};
```



Cada fila d'un array bidimensional és un array unidimensional.

`matriu` i `matriu[0]` fan referència a l'inici de la zona de dades, amb tipus de punter diferents.

`matriu` és un punter a punter a int

`matriu[0]` es un punter a int

4.9 Arrays multidimensionals

Sintaxi: `tipus nom[nombre_files][nombre_columnes];`

Exemple: `float altures[32][100];`

Assignació: `altures[3][45] = 1.75;`

Lectura: `scanf("%f", &altures[f][c]);`

El primer índex selecciona la fila i el segon la columna.

Per recòrrer un array bidimensional calen 2 bucles anidats.

`altures[0]`	identifica la primera fila
`altures[1]`	identifica la segona fila
`altures[0][0]`	és el primer element enter de la primera fila.

Exercici 

Escriu una funció `mitjanes()`, que donades les altures dels alumnes de diverses classes, calculi la mitjana de cada classe.

4.9 Arrays multidimensionals

Un array de cadenes es pot representar com un array bidimensional de tipus `char`, on cada fila conté una cadena de caràcters.

Sintaxis: `char noms[nombre_cadenes][longitud_maxima];`

Exemple: `char noms[3][6] = {"Maria", "Juan", "Jose"};`

Cada fila ha de tenir espai suficient per als caràcters i el caràcter fi de cadena '\0'.

4.9 Arrays multimensionals

Array tridimensional

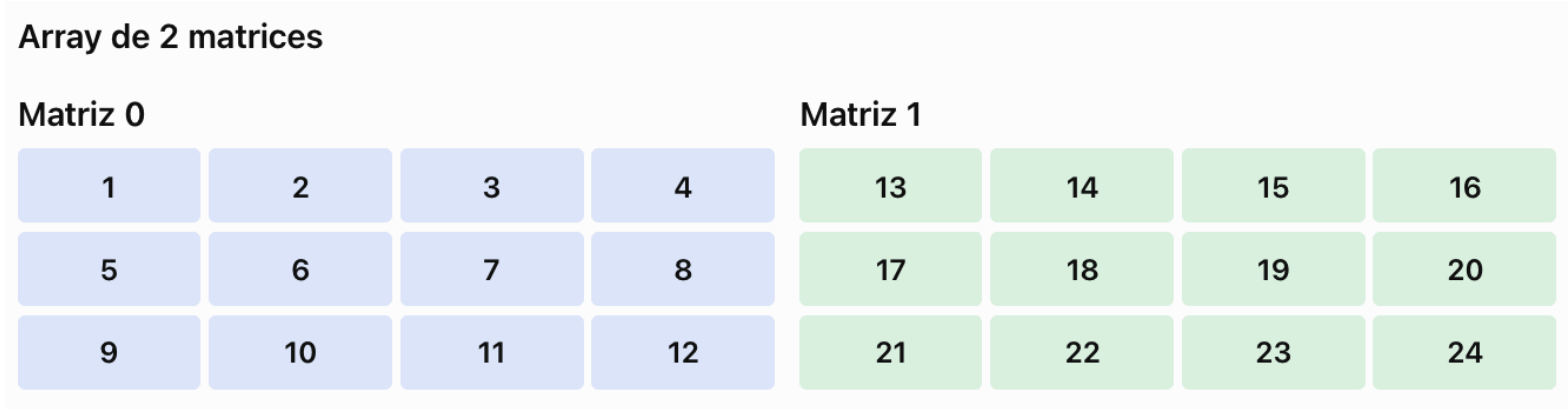
```
#define MATRICES 2
#define FILAS 3
#define COLUMNAS 4

int numeros[MATRICES][FILAS][COLUMNAS] = {

    {
        {1,  2,  3,  4},
        {5,  6,  7,  8},
        {9, 10, 11, 12}
    },

    {
        {13, 14, 15, 16},
        {17, 18, 19, 20},
        {21, 22, 23, 24}
    }

};
```



Per a recorrer-lo necessitem 3 iteratives aniuades

Exercici



La Facultat de Biologia està fent un estudi de la morfologia i del comportament de les formigues en els 4 continents següents: Amèrica, Àfrica, Àsia i Europa. Existeixen 3 espècies diferents de formigues: roja, marró i negra. Realitzeu un programa en C que emmagatzemi el nombre de potes que té cada espècie de formiga en cadascun dels continents. Utilitza enums per als continents (Continent) i formigues (Especie).

El programa haurà de:

- Utilitzar un array de cadenes de caràcters per guardar els noms dels 4 continents.
- Utilitzar un altre array de cadenes de caràcters per guardar els noms de les 3 espècies.
- Utilitzar un array bidimensional de 4 files i 3 columnes per guardar el nombre de potes.
- Inicialitzar les dades directament al programa, sense llegir-les pel teclat.
- Crear una funció que rebi l'array bidimensional i utilitzi assert() per comprovar que el nombre de potes de cada formiga està entre 0 i 6.
- Mostrar per pantalla el continent, l'espècie de formiga i el nombre de potes corresponent

Sortida per pantalla

```
A America, la formiga Roja te 6 potes.
A America, la formiga Marro te 6 potes.
A America, la formiga Negra te 6 potes.
A Africa, la formiga Roja te 6 potes.
A Africa, la formiga Marro te 5 potes.
A Africa, la formiga Negra te 6 potes.
A Asia, la formiga Roja te 6 potes.
A Asia, la formiga Marro te 6 potes.
A Asia, la formiga Negra te 4 potes.
A Europa, la formiga Roja te 6 potes.
A Europa, la formiga Marro te 6 potes.
A Europa, la formiga Negra te 6 potes.
```

Arrays en C vs llistes en python

Propiedad	Array de tamaño fijo en C	Lista en Python
Tamaño	Fijo	Dinámico
Tipo de datos	Todos los elementos del mismo tipo	Puede mezclar tipos
Modificar elementos	Sí	Sí
Añadir elementos	No directamente	Sí, con `append()`
Eliminar elementos	No directamente	Sí, con `remove()` o `pop()`
Memoria	Generalmente menor	Generalmente mayor
Gestión de memoria	Más explícita	Automática
Sintaxis de acceso	`array[0]`	`lista[0]`
Longitud	`sizeof(array)/sizeof(array[0])`	`len(lista)`